# SAVEETHA SCHOOL OF ENGINEERING SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:50
Duration : 1 Hour
**Industry Problem**

*Code of Conduct:*

*I* **PERAM VIVEKANANDA REDDY (192372330)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

*Signature of the student:*

Vivek

1. Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module sh ould accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

**2.** Convert the Pushdown Automata given below to a Context Free Grammar.

$M=(\{s_0,s_1\},\{a,b\},\{Z0,a\},\delta,s_0,Z_0,\phi)$ where $\delta$ is defined as

$\delta(s_0,a,Z_0)=\{(s_0,aZ_0)\}$

$\delta(s_0,a,a)=\{(s_0,aa)\}$

$\delta(s_0,b,a)=\{(s_1,\varepsilon)\}$

$\delta(s_1,b,a)=\{(s_1,\varepsilon)\}$

$\delta(s_1,\varepsilon,Z_0)=\{(s_1,\varepsilon)\}$

3. Design a Pushdown Automata for the context free grammar given below.Convert the CFG to a Pushdown Automata that accepts by empty stack. Write the formal definition of the PDA.

$S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$

4. Develop a **C-based computational module** to determine the **ε-closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ε-transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0\ A\ 1$
- $A \rightarrow 0\ A \mid 1\ A \mid \varepsilon$

| Criteria | Weightage | Q1 (10) | Q2 (10) | Q3 (10) | Q4 (10) | Q5 (10) | Total Marks (50) |
|---|---|---|---|---|---|---|---|
| Conceptual Understanding | 30 | 3 | 3 | 3 | 3 | 3 | 15 |
| Accuracy of Answer | 25 | 2.5 | 2.5 | 2.5 | 2.5 | 2.5 | 12.5 |
| Application of Knowledge | 20 | 2 | 2 | 2 | 2 | 2 | 10 |
| Completeness of Response | 15 | 1.5 | 1.5 | 1.5 | 1.5 | 1.5 | 7.5 |
| Clarity & Presentation | 10 | 1 | 1 | 1 | 1 | 1 | 5 |
| Total per Question | 10 Marks | /10 | /10 | /10 | /10 | /10 | / 50 |

## Question 1 — DFA-based C Validation Module

We need a DFA over the alphabet {a, b} that accepts exactly the strings that **begin with 'a' and end with 'a'**.

### *Design of the DFA*

States: Q0 (start, nothing read yet), A (last symbol read was 'a', reachable only if the first symbol was 'a'), B (last symbol read was 'b', first symbol was 'a'), and DEAD (trap state, used the moment the first symbol read is 'b'). State A is the only accepting state.

Transition Table:

| State | Input 'a' | Input 'b' |
|---|---|---|
| Q0 | A | DEAD |
| A | A | B |
| B | A | B |
| DEAD | DEAD | DEAD |

Start state : Q0
Accept state: A (only reached when the string is non-empty, begins with 'a' and its last symbol read is also 'a')

### *C Implementation*

```
#include <stdio.h>
```

```
#include <string.h>
typedef enum { Q0, A, B, DEAD
} State; int main() {
    char str[100];
    printf("Enter a string over {a, b}: ");
    scanf("%s", str);

    int len = strlen(str);
    State state = Q0;

    for (int i = 0; i < len; i++) {
        char ch = str[i];
        switch (state) {
            case Q0:
                state = (ch == 'a') ? A :
                DEAD; break;
            case A:
                state = (ch == 'a') ? A : B;
                break;
            case B:
                state = (ch == 'a') ? A : B;
                break;
            case DEAD:
                state = DEAD;
                break;
        }
```

```
if (len > 0 && state == A)
printf("Accepted\n");
    else
        printf("Rejected\n");

    return 0;
}
```

Sample runs: "a" □ Accepted, "aba" □ Accepted, "abba" □ Accepted, "ab" □ Rejected (does not end in 'a'), "baa" □ Rejected (does not begin with 'a'), "" (empty string) □ Rejected.

## Question 2 — Convert the PDA to a Context-Free Grammar

M = ({s0, s1}, {a, b}, {Z0, a}, □, s0, Z0, □), acceptance is by **empty stack** (the final-state set is empty).

delta(s0, a, Z0) = {(s0, aZ0)}
delta(s0, a, a ) = {(s0, aa )}
delta(s0, b, a ) = {(s1, e )}
delta(s1, b, a ) = {(s1, e )}
delta(s1, e, Z0) = {(s1, e )}  (e = epsilon)

### *Step 1 — Formal variable-based construction*

Introduce a variable A[p,q] for every pair of states p, q □ {s0, s1}. For a move that pops one symbol and pushes two symbols Y1Y2 (Y1 ends up on top), add A[p,q] □ a A[r,t] A[t,q] for every state t. For a pure pop move (nothing pushed) on input u that moves p to r, add A[p,r] □ u. Also include the base rule A[p,p] □ □ for every state p.

Applying this to the five transitions above (using shorthand A00 = A[s0,s0], A01 = A[s0,s1], A10 = A[s1,s0], A11 = A[s1,s1]) gives:

From delta(s0,a,Z0) and delta(s0,a,a) (both push two symbols, p = r = s0): A00 -> a A00 A00 | a A01 A10
A01 -> a A00 A01 | a A01 A11

From delta(s0,b,a) (pure pop, s0 -> s1, reads 'b'): A01 -> b

From delta(s1,b,a) (pure pop, s1 -> s1, reads 'b'): A11 -> b

From delta(s1,e,Z0) (pure pop, s1 -> s1, reads

epsilon): A11 -> e

Base rules:

A00 -> e

A11 -> e (already have this one)

A10 has no productions at all (no transition ever produces it), so it generates the empty language and any rule containing it is useless. Removing the useless alternative "A00 □ a A01 A10" and simplifying gives the reduced grammar:

A00 -> a A00 A00 | e (generates a*, i.e. any number of a's) A01 -> a A00 A01 | a A01 A11 | b
A11 -> b | e

***Step 2 — Start symbol and simplification***

Since acceptance is by empty stack, the start symbol of the grammar is S □ A[s0,s0] | A[s0,s1]. Tracing the PDA directly confirms what these variables generate: reading n copies of 'a' grows the stack to $a^n$ Z0, and it takes exactly n copies of 'b' to pop all the a's before the final □-move empties Z0. Hence the PDA accepts by empty stack exactly the strings $a^n$ $b^n$, n □ 1 (the empty string is not accepted, since s1

can only be entered after at least one 'a' has been pushed and popped).

### *Final Answer — Equivalent CFG*

S -> a S b | a b

Language generated: L = { a^n b^n : n >= 1

} Sample derivation for a^2 b^2 = "aabb":

S => aSb => a(ab)b => aabb

## Question 3 — CFG to PDA (Empty-Stack Acceptance)

Grammar:

S -> 0B | 1A
A -> 0 | 0S | 1AA
B -> 1 | 1S | 0BB

### *Standard top-down (empty-stack) construction*

For any CFG, an equivalent PDA that accepts by empty stack can be built using a **single state** q. The stack alphabet is the set of all terminals and non-terminals of the grammar, and the start stack symbol is the grammar's start symbol S. Every production A □ □ becomes an □-move that replaces A on the stack with □ (left symbol of □ ends up on top). Every terminal a on top of the stack is matched and removed by reading that same terminal from the input.

### *Formal definition of the PDA*

P = (Q, Sigma, Gamma, delta, q0, Z0, F)

Q = { q }
Sigma = { 0, 1 }
Gamma = { S, A, B, 0, 1 }
q0 = q
Z0 = S (initial stack symbol)
F = { } (empty stack acceptance -- no final states needed)

### *Transition function*

Expansion moves (epsilon input, replace non-terminal on top of stack):

delta(q, e, S) = { (q, 0B), (q, 1A) }
delta(q, e, A) = { (q, 0), (q, 0S), (q, 1AA) }
delta(q, e, B) = { (q, 1), (q, 1S), (q, 0BB) }

Matching moves (read input symbol, pop the same terminal from the stack): delta(q, 0, 0) = { (q, e) }
delta(q, 1, 1) = { (q, e) }

Working: the PDA starts with S on the stack. It repeatedly applies an expansion move (guessing the correct production, exactly mirroring a left-most derivation of the CFG) whenever a non-terminal is on top of the stack, and applies a matching move whenever a terminal is on top, consuming one input symbol. The input string is accepted if it is possible to empty the stack exactly when the entire input has been read.

Example: for the string "0011" the PDA can proceed as S □ 0B □ 0(0BB) □ 00(1)(1) = 0011, matching each derivation step to a stack expansion followed by matching moves that consume the '0's and '1's, ending with an empty stack.

## Question 4 — C Module for □-closure of NFA States

The □-closure of a state q is the set of all states reachable from q using zero or more □-transitions. We represent the □-transitions of the NFA as an adjacency matrix and compute the closure of every state with a depth-first search (DFS).

```
#include<stdio.h>
#include <string.h>

#define MAX_STATES 20

int epsilonTrans[MAX_STATES][MAX_STATES]; /*   epsilonTrans[i][j]
= 1 if i -e-> j */ int visited[MAX_STATES];
int n;     /* number of states in the NFA */

/* Recursive DFS to build the epsilon-closure of
'state' */ void epsilonClosure(int state, int closure[],
int *count) {
    if  (visited[state])  return;
    visited[state]      =      1;
    closure[(*count)++]     =
    state;

    for (int j = 0; j < n; j++) {
        if (epsilonTrans[state][j] && !visited[j])
            epsilonClosure(j, closure, count);
    }
}

int main() {
    int edges;

    printf("Enter   number   of   states:   ");
    scanf("%d", &n);

    memset(epsilonTrans, 0, sizeof(epsilonTrans));

    printf("Enter number of epsilon-transitions:
    "); scanf("%d", &edges);
    printf("Enter   each   transition   as:   from
    to\n"); for (int i = 0; i < edges; i++) {
```

```
        int f, t;
        scanf("%d %d", &f,
        &t);
        epsilonTrans[f][t] =
        1;
    }

    /* Compute and print epsilon-closure for every
    state */ for (int s = 0; s < n; s++) {
        int closure[MAX_STATES], count
        =       0;       memset(visited,       0,
        sizeof(visited));

        epsilonClosure(s, closure, &count);

        printf("epsilon-closure(%d) = { ",
        s); for (int i = 0; i < count; i++)
            printf("%d ", closure[i]);
        printf("}\n");
    }

    return 0;
}
```

This module underpins NFA-to-DFA subset construction used in compiler lexical analysers and regular-expression pattern-matching engines: for every DFA state (a set of NFA states) the □-closure

must be applied after every symbol move.

## Question 5 — C Module to Validate a Binary String against a CFG

Grammar:

S -> 0 A 1
A -> 0 A | 1 A | e  (e = epsilon)

### *Deriving the language*

A □ 0A | 1A | □ generates any (possibly empty) string of 0's and 1's, i.e. A generates (0|1)*. Substituting this into S □ 0 A 1 gives the language L(G) = 0 (0|1)* 1 — every string that **starts with 0, ends with 1, and has length □ 2**, with anything (0's or 1's) allowed in between.

### *C Implementation*

```
#include<stdio.h>
#include <string.h>

/* Validates str against S -> 0 A 1 , A -> 0A | 1A | epsilon
   i.e. checks membership in L = 0 (0|1)* 1                    */
int isValid(char *str) {
    int len = strlen(str);

    /* Minimum derivation is S -> 0 A 1 with A -> epsilon, i.e.
    "01" */ if (len < 2) return 0;

    /* First symbol must be matched by the leading '0' in S -> 0
    A 1 */ if (str[0] != '0') return 0;

    /* Last symbol must be matched by the trailing '1' in S -> 0
    A 1 */ if (str[len - 1] != '1') return 0;

    /* Every symbol strictly between the first and last must
       come from A -> 0A | 1A | epsilon, i.e. it must be 0 or 1 */
    for (int i = 1; i < len - 1; i++) {
        if (str[i] != '0' && str[i] != '1') return 0;
    }

    return 1;
}
```

```
int main() {
    char str[1000];
    printf("Enter a binary string:
    "); scanf("%s", str);

    if (isValid(str))
        printf("Accepted -- string conforms to the CFG (S -> 0A1, A -> 0A|1A|e)\n");
    else
        printf("Rejected\n");

    return 0;
}
```

Sample runs: "01" Accepted (n=0 middle), "0101" Accepted, "0111001" Accepted, "1" Rejected (too short / no leading 0), "00" Rejected (does not end in 1), "10" Rejected (does not start with 0).